



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Corso di Big Data Engineering

Clinical report AI Assistant

Anno Accademico 2024/2025

Componenti del gruppo

Gennaro Iannicelli

matr. M63001668

Giuseppe Gatta

matr. M63001669

Aurora D'Ambrosio

matr. M63001662

Contents

1	Panoramica del sistema	1
1.1	Introduzione	1
1.1.1	Un supporto al personale sanitario	2
1.1.2	Obiettivi	2
1.2	Strumenti utilizzati	3
1.2.1	Voice to text - Whisper	3
1.2.2	LLM - Mistral 7B	3
1.2.3	Python	4
1.2.4	Database - MongoDB e Redis	4
1.2.5	NER - Named Entity Recognition	5
1.2.6	Frontend - Streamlit	5
2	Specifiche del sistema	6
2.1	Specifica informale	6
2.1.1	User stories	7
2.2	Specifiche formali	7
2.2.1	Use Case Diagram	9
2.3	Dalle specifiche al design	12
2.3.1	Architettura del sistema	12

2.3.2	Block Definition Diagram	16
2.4	Design dinamici del sistema	17
2.4.1	Activity diagram	17
2.5	Sequence diagram	22
3	Implementazione del sistema	25
3.1	Architettura a microservizi	25
3.2	LLM Pipeline	26
3.2.1	Transcription Pipeline	26
3.2.2	NER e LLM	27
3.3	Database	33
3.3.1	Popolamento del Database	35
3.3.2	Analytics per l'Amministratore	35
3.4	Dashboard	37
3.5	Dockerization	39
3.5.1	Dockerizzazione del sistema	39
3.5.2	Struttura del Dockerfile	40
3.5.3	Docker Compose: orchestrazione multi servizio	40
4	Testing	42
4.1	Integration testing	43
4.1.1	Test di Pipeline Manager	43
4.1.2	Test del Database	45
4.2	UI Testing	45
5	Conclusioni	47

Chapter 1

Panoramica del sistema

1.1 Introduzione

La crescente necessità di strumenti digitali negli ambienti clinici ha stimolato lo sviluppo di soluzioni a supporto del personale sanitario nelle attività amministrative e di refertazione. Uno dei processi più critici e dispendiosi in termini di tempo è la creazione dei referti clinici. Il progetto “Medical Voice-to-Text System for Clinical Report Automation” è stato sviluppato per semplificare e velocizzare il processo di compilazione dei referti clinici da parte del personale sanitario. Utilizzando l’elaborazione vocale (Whisper), modelli linguistici avanzati (Mistral 7B) e una struttura dati efficiente basata su MongoDB e Redis, il sistema consente di trasformare la voce dei medici in referti strutturati e modificabili.

1.1.1 Un supporto al personale sanitario

La documentazione clinica tradizionale si basa pesantemente sull'inserimento manuale da parte del personale medico, consumando tempo prezioso e introducendo il rischio di errori. Strumenti di trascrizione vocale sono stati adottati in alcuni contesti sanitari, ma molti mancano dell'accuratezza, del supporto linguistico e della comprensione contestuale necessari. Con l'avvento dei Large Language Models (LLM) e dei sistemi di riconoscimento vocale migliorati, è ora possibile costruire soluzioni più intelligenti, multilingue e integrate, che assistano il personale sanitario in tempo reale. Questo progetto affronta tali sfide combinando trascrizione, comprensione linguistica e gestione strutturata dei dati.

1.1.2 Obiettivi

Gli obiettivi cui si è cercato di raggiungere con l'implementazione del sistema sono i seguenti:

- Automatizzare la trascrizione vocale in testo medico.
- Generare report clinici coerenti, sintetici e strutturati
- Permettere la modifica o cancellazione dei dati clinici
- Ridurre il tempo speso in attività burocratiche.
- Migliorare l'accuratezza e la standardizzazione dei referti.

1.2 Strumenti utilizzati

Veloce panoramica degli strumenti utilizzati per l'implementazione del sistema, verranno trattati più a fondo nei prossimi capitoli del documento

1.2.1 Voice to text - Whisper

Whisper è un avanzato modello di riconoscimento vocale automatico (ASR) sviluppato da OpenAI. Grazie alla sua capacità di comprendere con elevata accuratezza diversi accenti, rumori di fondo e terminologia tecnica, viene utilizzato per convertire le dettature mediche in testo scritto.

1.2.2 LLM - Mistral 7B

Mistral 7B è un modello linguistico di grandi dimensioni (LLM), leggero ma altamente performante. Integrato tramite Ollama, è in grado di elaborare le trascrizioni testuali generate da Whisper per estrarre automaticamente informazioni cliniche rilevanti. Questo processo avviene attraverso una pipeline RAG (Retrieval-Augmented Generation), che combina la generazione di testo con il recupero intelligente di dati da fonti strutturate, permettendo analisi più precise e contestualizzate.

1.2.3 Python

L'intera logica applicativa del sistema è sviluppata in Python, un linguaggio noto per la sua chiarezza sintattica e ampia disponibilità di librerie. Il backend si occupa di gestire le richieste dell'utente, orchestrare i vari moduli (ASR, LLM, database), processare i dati clinici e garantire l'integrità delle operazioni eseguite, fungendo da cuore del sistema.

1.2.4 Database - MongoDB e Redis

MongoDB

Per la gestione dei dati clinici strutturati e semi-strutturati, viene utilizzato MongoDB, un database NoSQL orientato ai documenti. Questo permette di archiviare in modo flessibile le trascrizioni, i riferimenti ai file audio e le informazioni anagrafiche dei pazienti, organizzandole per medico e facilitando un accesso rapido e scalabile.

Redis

Redis viene utilizzato come sistema di caching ad alte prestazioni per ottimizzare l'interazione in tempo reale tra la dashboard e il backend. In particolare, memorizza temporaneamente informazioni sui pazienti e risultati intermedi per ridurre la latenza nelle risposte e migliorare l'esperienza utente.

1.2.5 NER - Named Entity Recognition

È una tecnica di elaborazione del linguaggio naturale (NLP) grazie al quale è possibile identificare e classificare entità nominate come persone, organizzazioni, luoghi, date e così via. Nel contesto specifico viene utilizzato per riconoscere termini medici relativi a diagnosi e patologie all'interno delle trascrizioni del medico.

1.2.6 Frontend - Streamlit

Il frontend dell'applicazione è sviluppato con Streamlit, un framework Python pensato per la creazione rapida di interfacce web interattive. Viene utilizzato per offrire una dashboard semplice e intuitiva, attraverso cui il personale medico può visualizzare le trascrizioni, monitorare i dati dei pazienti e interagire con il sistema in modo immediato e funzionale.

Chapter 2

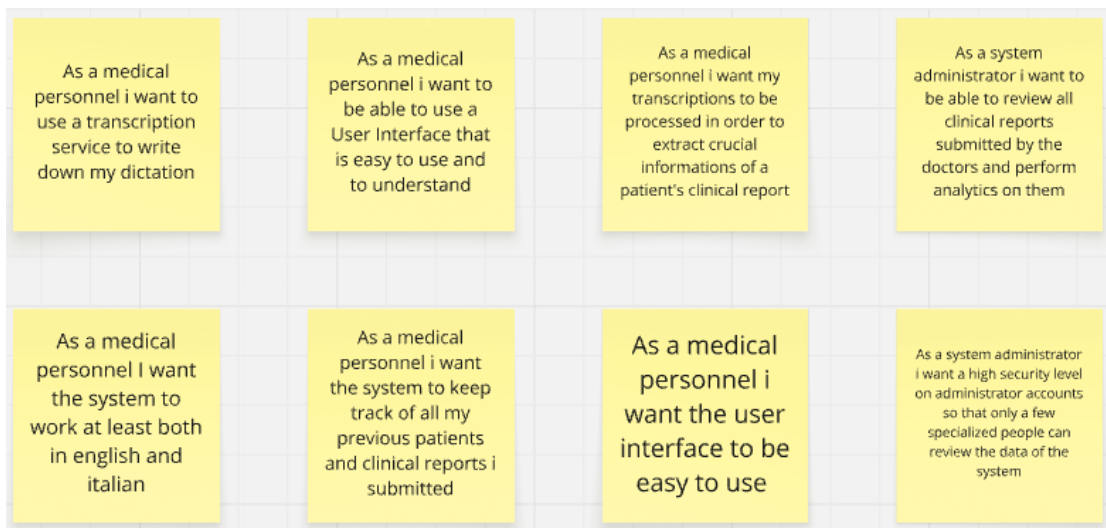
Specifiche del sistema

2.1 Specifica informale

Il sistema consente al personale medico effettuare registrazioni della propria voce. L'audio viene poi trascritto, anonimizzato, elaborato da un LLM che ne estrae i dati clinici strutturati ed infine il risultato viene visualizzato nella dashboard e salvato nella base di dati. Gli utenti possono revisionare, modificare, cercare e scaricare i referti clinici. Il sistema prevede anche una sezione amministrativa, dedicata agli admin dell'applicativo, in cui è possibile effettuare analitiche sui dati presenti nella base di dati. Il sistema prevede un meccanismo di protezione della privacy dei pazienti attraverso un meccanismo di anonimizzazione dei dati sensibili forniti in ingresso al LLM. Il sistema supporta trascrizioni in lingua inglese e italiana, garantendo usabilità e un ampio target di utenti che utilizzano il sistema.

2.1.1 User stories

Le user stories sono brevi descrizioni di una funzionalità dal punto di vista dell'utente. Sono utilizzate per catturare le esigenze degli utenti e avere una migliore comprensione di quale è il focus durante il processo di implementazione del sistema. Le user stories definite per il sistema sono le seguenti:



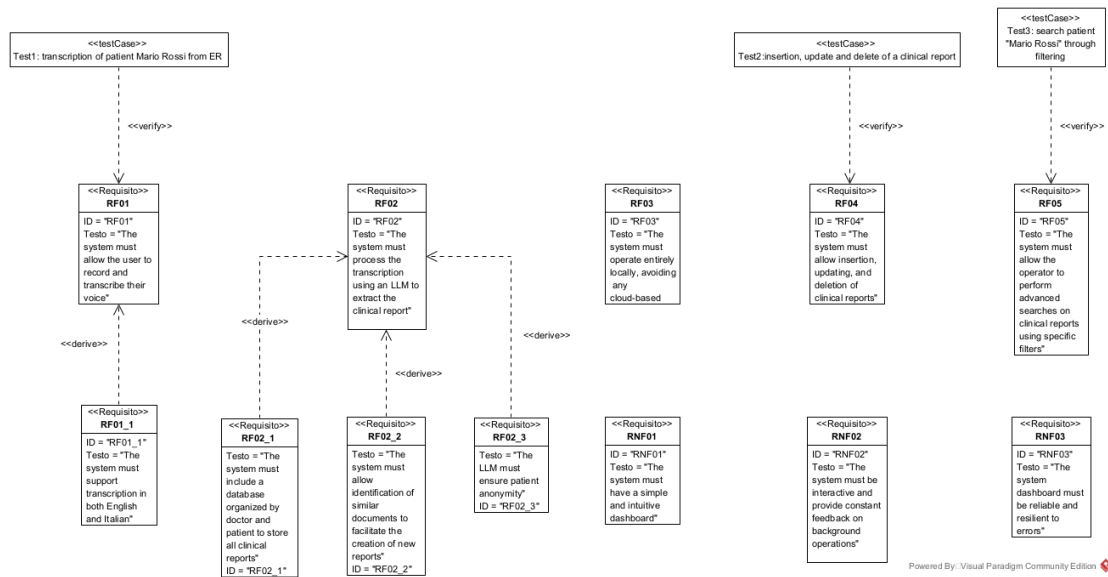
2.2 Specifiche formali

Dalla specifica informale, nascono i seguenti requisiti, funzionali e non:

- RF01: Il sistema deve consentire all'utente di registrare e trascrivere la propria voce
- RF02: Il sistema deve processare la trascrizione tramite un sistema di LLM per estrarne il referto clinico

- RF03: Il sistema deve operare esclusivamente in locale, evitando soluzioni cloud - based
- RF04: Il sistema deve consentire l'inserimento, modifica, cancellazione e download dei referti clinici
- RF05 Il sistema deve consentire agli utenti di effettuare operazioni di ricerca avanzate sui referti clinici tramite specifici filtri
- RNF01: Il sistema deve avere una dashboard semplice ed intuitiva
- RNF02: Il sistema deve essere interattivo e garantire feedback costante sulle operazioni in background
- RNF03: Il sistema deve disporre di una dashboard affidabile e priva di errori

Dai requisiti soprastanti nasce il seguente requirement diagram, contenente anche un insieme di sottorequisiti scaturiti dai requisiti padre.



2.2.1 Use Case Diagram

Lo use case diagram mostra le interazioni tra attori e sistema, fornendo una panoramica delle funzionalità del sistema. Tali diagrammi sono fondamentali per comprendere l'implementazione del sistema. Nel sistema sono stati definiti 2 attori principali.

Healthcare Personnel (Personale sanitario, medico, infermiere etc.)

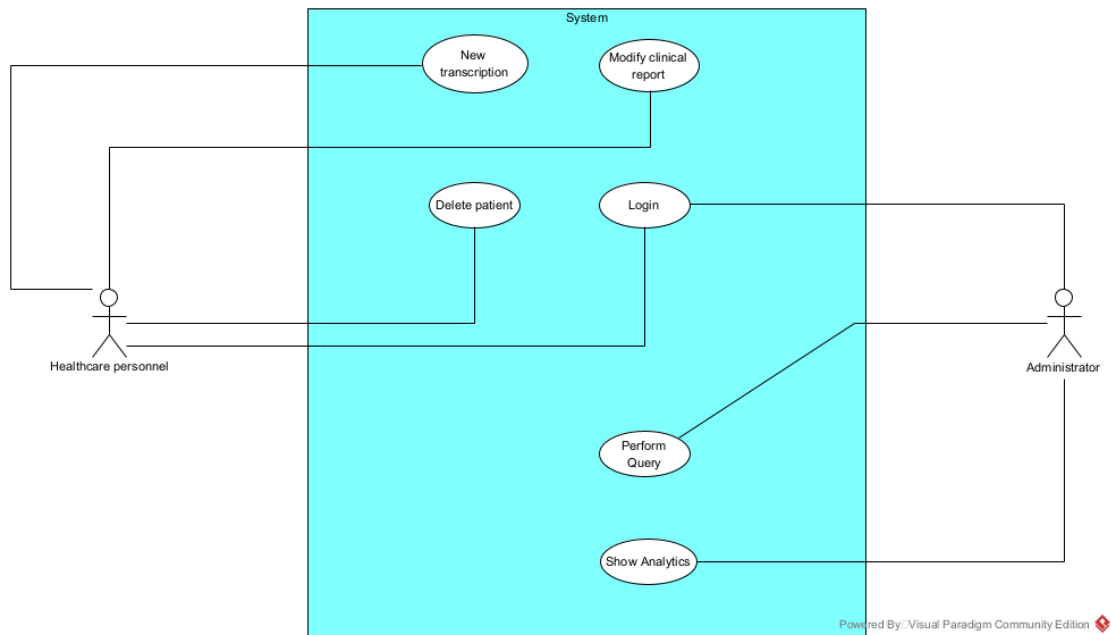
Il personale sanitario utilizza il sistema principalmente per la gestione e la creazione dei referti clinici. Attraverso l'interfaccia dedicata, può registrare nuove trascrizioni vocali che vengono poi trasformate in referti, aggiornare quelli già esistenti per correggere o aggiungere informazioni, e cancellare i dati dei pazienti quando necessario, ad esempio in caso di dimissioni o errori. L'accesso al sistema avviene tramite un

meccanismo di login sicuro. Inoltre, il personale ha la possibilità di eseguire interrogazioni all'interno della piattaforma per cercare referti specifici o filtrare quelli già generati.

Administrator (Amministratore)

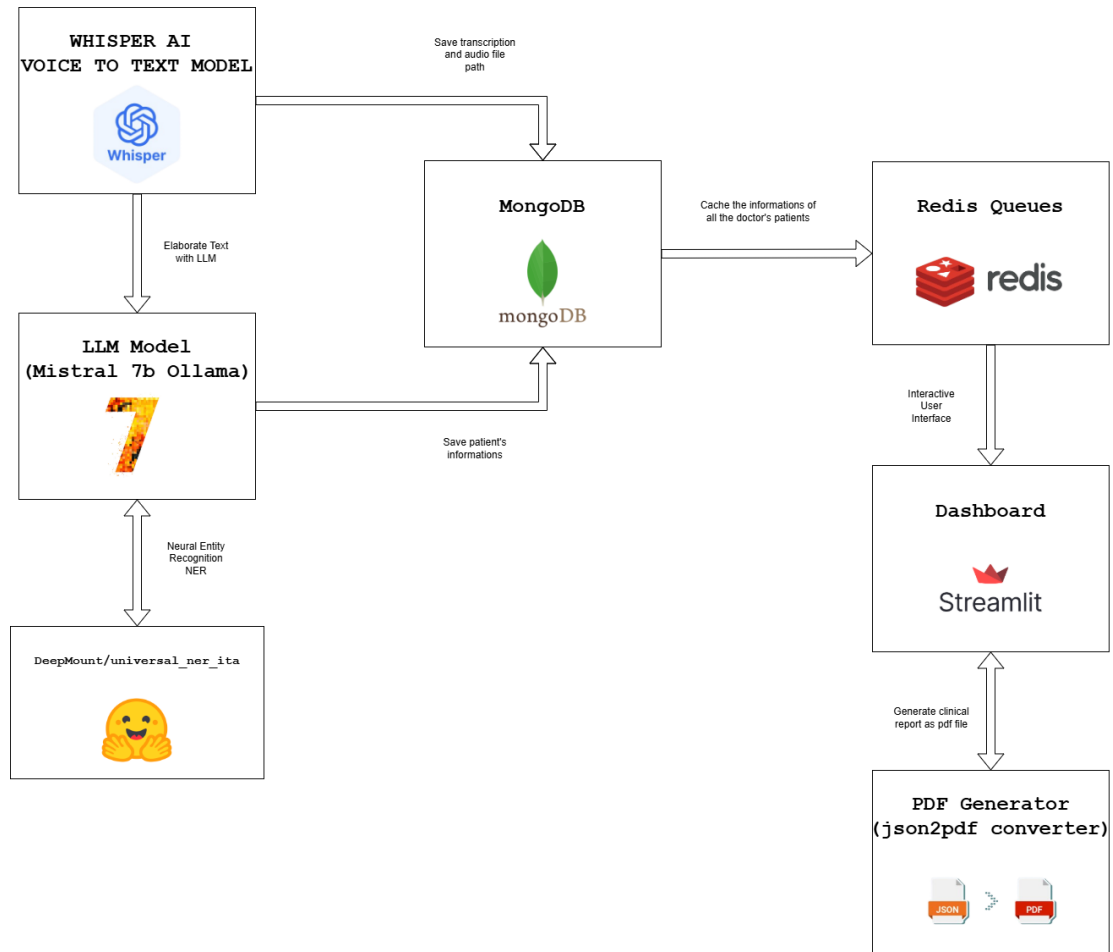
La figura di amministratore del sistema viene introdotta nel contesto dei Big Data per consentire elaborazione e studio dei dati acquisiti nel sistema. Dopo l'accesso tramite login, può eseguire interrogazioni complesse sui dati contenuti nel sistema, allo scopo di monitorare l'efficienza e l'utilizzo della piattaforma. A sua disposizione sono presenti strumenti di analytics che consentono di visualizzare report aggregati e statistiche, utili per individuare eventuali criticità, ottimizzare i processi e scovare eventuali colli di bottiglia nell'affluenza ai reparti. Il suo intervento è fondamentale per garantire un buon grado di efficienza nelle operazioni svolte dall'ospedale. L'amministratore è un ruolo con un alto grado libertà sull'applicativo, per questo motivo è legato ad un forte livello di sicurezza nel sistema: un nuovo amministratore può essere definito solo a monte di un processo decisionale che scaturisce nella registrazione di un nuovo profilo da parte di uno degli amministratori già esistenti.

CHAPTER 2. SPECIFICHE DEL SISTEMA



2.3 Dalle specifiche al design

2.3.1 Architettura del sistema



Il sistema si compone dei seguenti macro componenti principali:

- LLM Pipeline: gestisce la pipeline di generazione dei referti, è a sua volta composta dai seguenti blocchi
 - Whisper: modello di trascrizione da voce a testo, trasforma file audio in trascrizioni testuali

- NER: Named Entity Recognition per l'estrazione di termini rilevanti all'interno delle trascrizioni mediche
- Mistral 7B di Ollama: modello LLM per la generazione del referto clinico a partire dalla trascrizione
- Database: il database gestisce la persistenza dei dati, garantendo sicurezza e privacy sui dati dei pazienti e sui profili dei medici. Il database si compone di due componenti:
 - MongoDB: database NoSQL document oriented
 - Redis: sistema a code che viene utilizzato per cachare i dati contenuti in MongoDB
- Dashboard Streamlit: dashboard interattiva che consente l'utilizzo del sistema ai diversi utenti (medici e amministratori).

Seguirà una descrizione più dettagliata dei macroblocchi.

Fronted (Streamlit)

L'interfaccia utente è sviluppata in Streamlit. Questa componente è accessibile al personale medico e consente di registrare file audio, avviare il processo di trascrizione e generazione del referto clinico, nonché visualizzare, modificare o cancellare i referti già esistenti. Durante il processo di generazione del referto, la dashboard consente di visualizzare il documento appena prodotto al fine di validarne i dati; in particolare, all'utente è richiesto di visualizzare e validare, previa

mancato completamento della pipeline, le anagrafiche del paziente e di completare l'eventuale referto parzialmente incompleto. Attraverso un sistema di generazione di file, il sistema consente al medico di scaricare i referti dei propri pazienti in formato PDF. All'utente base è infine consentito effettuare operazioni di filtraggio sui dati al fine di ottenere ed eventualmente studiare lo storico dei propri pazienti. Lato admin invece, la dashboard dispone di due funzionalità fondamentali: delle operazioni di filtraggio sui dati, analoghe a quelle dell'utente base, e una sezione dedicata alle analytics, in cui i referti sui pazienti vengono aggregati ed elaborati al fine di ottenere dei grafici esplicativi dell'andamento dell'ospedale. Le interazioni con il sistema vengono tradotte in richieste API che vengono gestite dal controller nel backend.

Componenti per architettura a microservizi

Controller, backend e chiamate API rappresentano il cuore logico del sistema, responsabili della gestione e coordinamento delle principali funzionalità applicative. È composto da tre moduli fondamentali: Controller, Backend e API_controller:

- Controller: Riceve e gestisce le richieste provenienti dal frontend tramite API_Request come la creazione, la cancellazione o la visualizzazione di referti ed inoltra queste richieste, in accordo con il funzionamento previsto da un'architettura a microservizi,

al blocco Backend che ne implementa il funzionamento vero e proprio.

- Backend: Esegue le operazioni ricevute dal controller e si occupa della gestione diretta del flusso di dati verso il database e la pipeline di LLM. Offre funzionalità come la generazione e cancellazione dei referti.

LLM Pipeline

La pipeline di LLM implementa la funzione principale di generazione dei report. Dopo aver acquisito la trascrizione audio generata dalla dashboard, si procede con l'estrazione dell'anagrafica del paziente; ciò è reso possibile dall'utilizzo combinato di un NER e di pattern RegEx, e al calcolo di un ID univoco che consentirà l'identificazione del referto all'interno del database. In seguito, la pipeline procederà ad anonimizzare il referto così da poterlo fornire alla LLM per la generazione di una scheda di ammissione al pronto soccorso o referto clinico strutturato sulla base della modalità di funzionamento scelta dall'utente. La pipeline procederà anche al salvataggio sia delle trascrizioni che dei referti generati.

Database

Il sistema utilizza MongoDB come database principale, strutturato in tre collezioni:

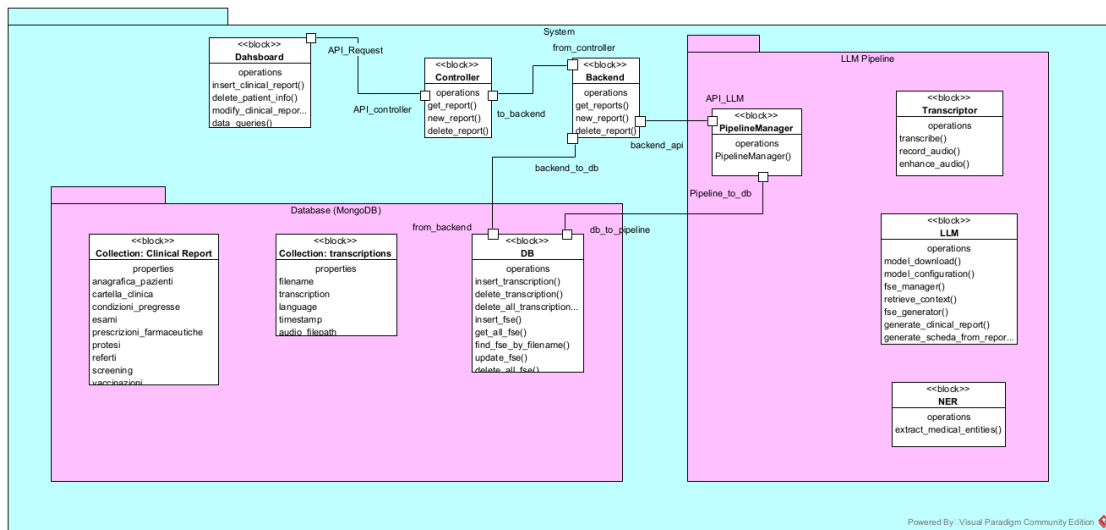
- Clinical Report: Contiene tutti i dati strutturati relativi ai referti clinici generati.
- Transcriptions: Memorizza le trascrizioni vocali generate dal sistema, con proprietà come nome del file, testo trascritto, lingua, timestamp e percorso del file audio.
- Medical Operators: Contiene tutti i dati dell'utente tra cui anagrafica, ruolo e dati dell'ospedale di appartenenza.

2.3.2 Block Definition Diagram

Un Block Definition Diagram (BDD), o diagramma di definizione del blocco, è un diagramma di modellazione strutturale che visualizza la struttura statica di un sistema, i suoi componenti, le loro relazioni, le loro proprietà, le interfacce e i vincoli. È un diagramma statico che descrive la struttura di un sistema o di un sottosistema, mostrando come i blocchi si collegano tra loro. Dalla definizione dell'architettura a microservizi vista pocanzi è nato il seguente BDD, in cui è chiara la separazione di responsabilità e la modularità. In particolare distinguiamo i seguenti macroblocchi:

- Dashboard: componente grafico
- Controller e backend: implementano, attraverso richieste API, la struttura a microservizi del sistema

- LLM Pipeline: organizza ed orchestra tutta la pipeline di machine learning necessaria per la gestione dei referti clinici, a partire dalla loro creazione
- Database: mongoDB e code Redis per il caching dei dati



2.4 Design dinamici del sistema

2.4.1 Activity diagram

Un activity diagram è un tipo di diagramma che visualizza il flusso di attività all'interno di un sistema o di un processo. Questi diagrammi, spesso utilizzati nella modellazione di software o processi aziendali, rappresentano in modo visivo le azioni che vengono eseguite, le decisioni che vengono prese e le relazioni tra queste attività.

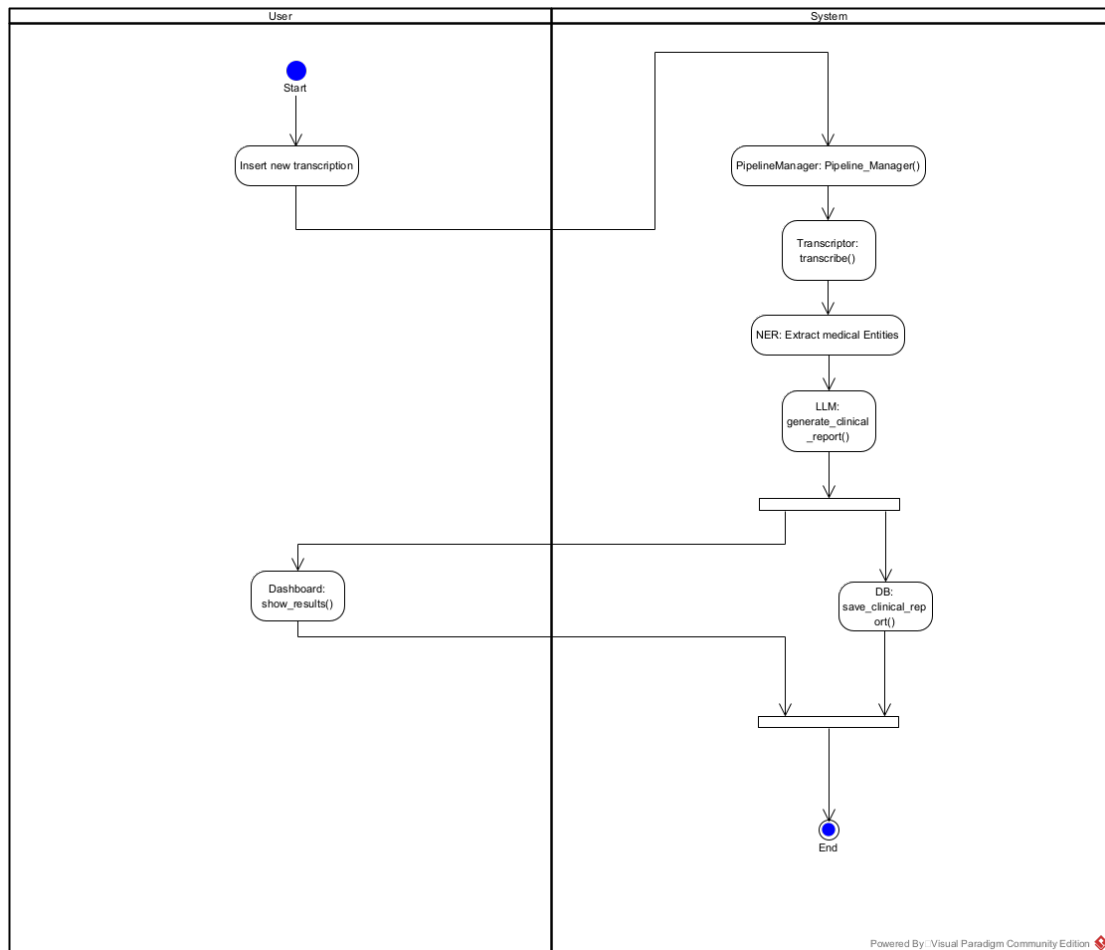
Activity 1: creazione nuovo referto

Il diagramma rappresenta il flusso automatico che si attiva quando un medico desidera generare un referto a partire da una registrazione vocale. Il processo è suddiviso tra le azioni dell'utente (a sinistra) e i moduli del sistema (a destra), e si compone delle seguenti fasi:

1. Caricamento audio da interfaccia: Il processo inizia con l'interazione da parte del medico con il sistema. Attraverso un apposito bottone, l'utente registra un file audio che viene poi mandato nel backend per l'avvio della pipeline di trascrizione e elaborazione del testo.
2. Avvio della PipelineManager: Il file audio viene inoltrato al modulo PipelineManager, che ha il compito di coordinare le diverse fasi di elaborazione linguistica, orchestrando i moduli di trascrizione, estrazione semantica e generazione testuale.
3. Trascrizione automatica con Whisper: Il modulo Transcriptor prende in carico il file e ne esegue la trascrizione automatica sfruttando faster Whisper, un modello open-source di deep learning per la trascrizione vocale. Il risultato è un testo trascritto in linguaggio naturale.
4. Estrazione di entità cliniche: Una volta ottenuto il testo trascritto, il sistema passa alla fase di riconoscimento di entità mediche. Questo è svolto dal modulo NER (Named Entity Recognition),

che isola i concetti rilevanti dal punto di vista clinico (es. patologie, farmaci, dati anagrafici, condizioni pregresse).

5. Generazione del referto con Mistral 7B: Il contenuto elaborato viene poi fornito al modulo LLM, che utilizza il modello Mistral 7B per produrre un referto clinico strutturato. Il sistema può inoltre arricchire il contesto informativo tramite una tecnica di RAG (Retrieval-Augmented Generation), recuperando dati da documentazione clinica pregressa o simili referti presenti nel database.
6. Salvataggio del referto: Una volta generato, il referto viene salvato nel database tramite la funzione `save_clinical_report()` del modulo DB. In questa fase vengono aggiornate le collezioni Clinical Report e Transcriptions del database MongoDB.
7. Visualizzazione del risultato: terminato il processo, l'utente può visualizzare il referto direttamente sull'interfaccia, grazie alla funzione `show_results()` del modulo Dashboard.

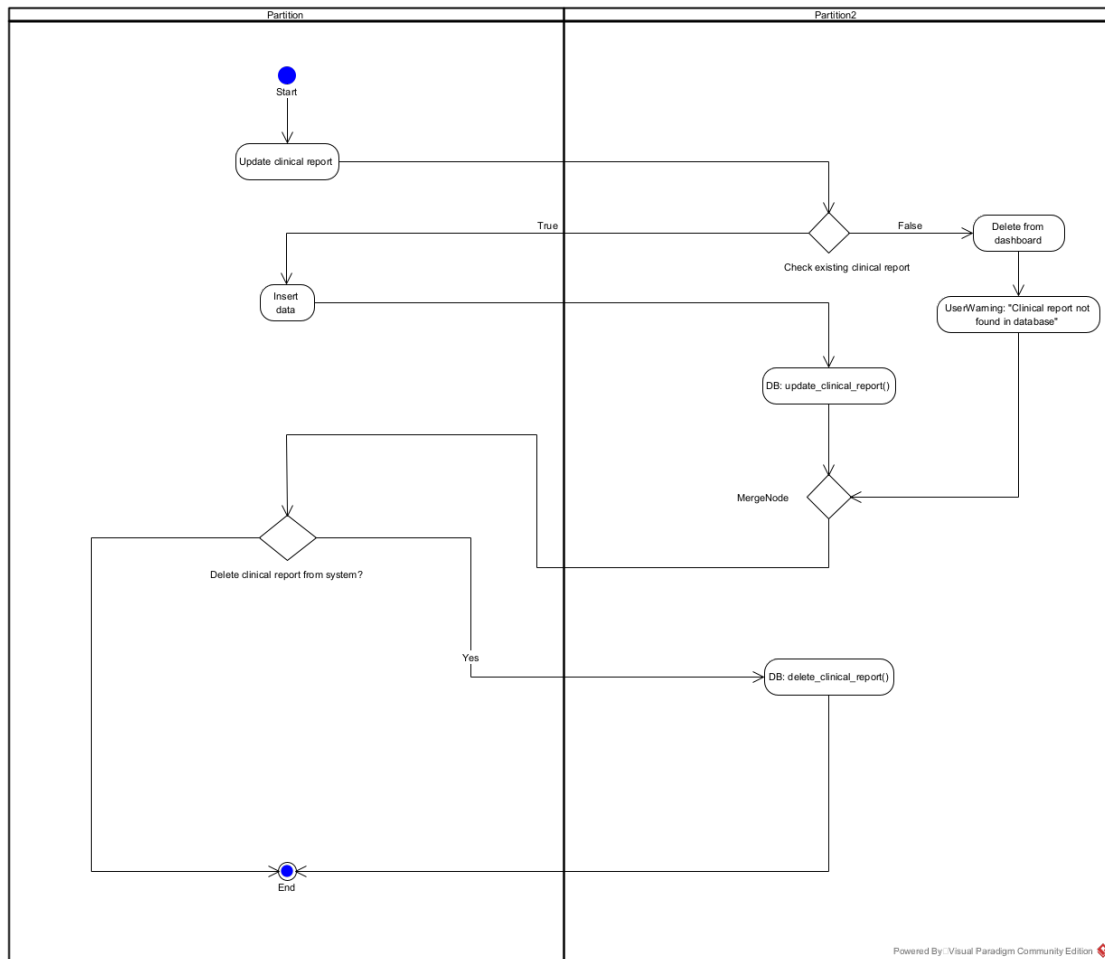


Activity 2: Aggiornamento e cancellazione referto

Questo diagramma descrive il flusso di aggiornamento o eliminazione di un referto clinico, suddividendo le attività tra due partizioni distinte: Partition, che rappresenta le azioni svolte dall'utente o dal sistema a livello applicativo, e Partition 2, che include le operazioni eseguite sul database o relative alla gestione interna dei dati. Il flusso delle azioni da eseguire può essere racchiuso in questi passaggi:

- Il processo inizia con l'azione "Update clinical report", ovvero la richiesta di aggiornamento di un referto clinico.

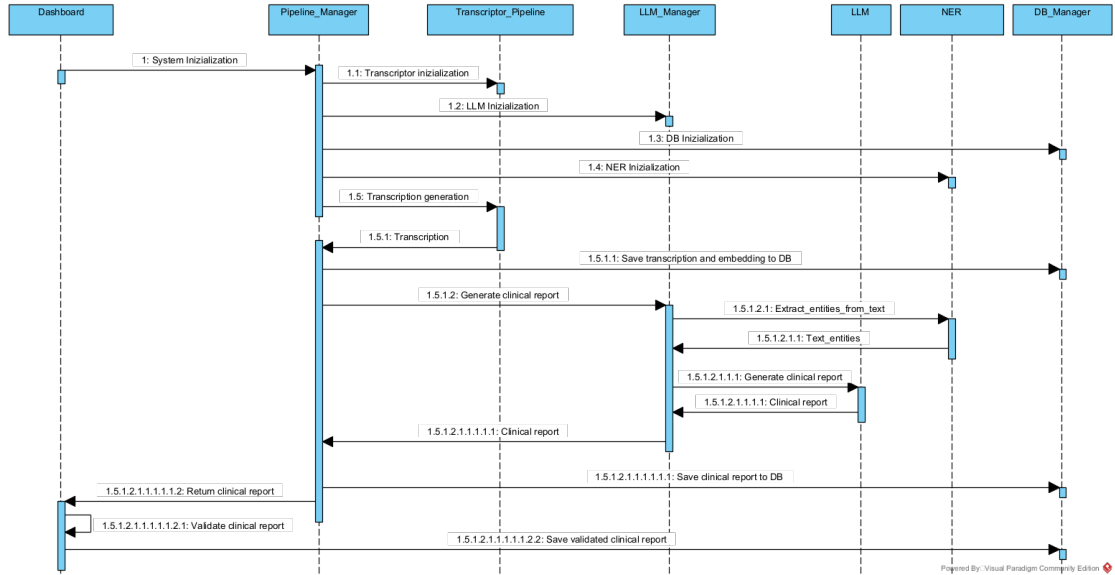
- Subito dopo, il sistema verifica se il referto esiste effettivamente nel database (“Check existing clinical report”). Se il referto non esiste, il sistema lo elimina dalla IO (“Delete from dashboard”) e mostra un messaggio di avviso all’utente: “Clinical report not found in database”. Se il referto esiste, vengono inseriti i nuovi dati e il referto clinico viene aggiornato nel database attraverso la funzione `update_clinical_report`.
- Proseguendo si esplora la possibilità di eliminare il report clinico dal sistema: se l’utente preme sul bottone di eliminazione, il sistema procede con l’eliminazione del referto dal database attraverso la funzione `delete_clinical_report`



2.5 Sequence diagram

Un diagramma di sequenza (in inglese Sequence Diagram) è un diagramma previsto dall'UML utilizzato per descrivere uno scenario. Uno scenario è una determinata sequenza di azioni in cui tutte le scelte sono state già effettuate; in pratica nel diagramma non compaiono scelte, né flussi alternativi. Normalmente da ogni diagramma di attività sono derivati uno o più diagrammi di sequenza; se per esempio il diagramma di attività descrivesse due flussi di azioni alternativi, se ne potrebbero

ricavare due scenari, e quindi due diagrammi di sequenza alternativi. Di seguito descriviamo il sequence diagram scaturito dall'activity diagram n.1, relativo alla generazione di un nuovo referto clinico.



Il processo di trascrizione vocale nel sistema prevede un'interazione fluida tra l'utente e una serie di componenti software integrate, che lavorano in modo sinergico per trasformare la dettatura medica in un report clinico strutturato e consultabile. L'utente, ovvero il personale medico, inizia dettando un contenuto clinico, che viene acquisito e trascritto automaticamente grazie al modello di riconoscimento vocale Whisper AI. Il risultato della trascrizione, insieme al file audio originale, viene immediatamente memorizzato in una collection di MongoDB, che funge da archivio centrale delle informazioni. Una volta completata la trascrizione, il testo viene elaborato da un modello linguistico di grandi dimensioni (LLM), in questo caso Mistral 7B eseguito tramite Ollama, il quale si occupa di analizzare e comprendere

il contenuto medico. A supporto di questa fase interviene un modello specializzato di Named Entity Recognition (NER), che consente di identificare all'interno del testo entità cliniche rilevanti, come sintomi, farmaci, diagnosi o parametri vitali. Queste informazioni strutturate vengono poi reinserite nel database, arricchendo il profilo del paziente con dati utili per il monitoraggio e la continuità delle cure. Parallelamente, le informazioni archiviate vengono temporaneamente memorizzate in una coda Redis, che ha il compito di ottimizzare le prestazioni del sistema garantendo un accesso rapido ai dati da parte dell'interfaccia utente. L'utente finale interagisce infatti attraverso una dashboard realizzata con Streamlit, che consente di visualizzare i report trascritti, analizzare le informazioni cliniche estratte, e navigare tra i dati dei diversi pazienti. Dalla stessa interfaccia, è possibile generare automaticamente report in formato PDF grazie a un modulo di conversione da JSON a PDF, utile per l'archiviazione documentale o la condivisione formale delle informazioni.

Chapter 3

Implementazione del sistema

3.1 Architettura a microservizi

Il sistema è progettato secondo un'architettura modulare e event-driven, con un'attenzione particolare alla separazione delle responsabilità tra i componenti. Questo approccio consente ai diversi moduli di operare in modo indipendente e coordinato, garantendo scalabilità, flessibilità nell'evoluzione del sistema e resilienza agli errori o malfunzionamenti locali. Nello specifico, l'architettura si ispira al pattern delle architetture a microservizi, separando chiaramente le responsabilità funzionali in moduli indipendenti ed intercambiabili.

3.2 LLM Pipeline

3.2.1 Transcription Pipeline

Questo componente ha il compito di generare trascrizioni del dettato medico, ottenuto attraverso il file audio che viene registrato dalla Dashboard. La pipeline di trascrizione si compone di due blocchi:

- Modulo di preprocessing dell'audio: prima di essere passato al modello di trascrizione, il file audio viene elaborato da un sistema di miglioramento dell'audio che applica su di esso un insieme di filtri attraverso i quali si riesce a garantire un'ulteriore accuratezza nel passaggio da voce a testo:
 - Filtro passa banda: Isola solo una gamma di frequenze (es. voce umana), eliminando suoni troppo bassi o troppo alti.
 - Filtro passa basso: Mantiene le frequenze gravi (es. rumori profondi), attenua suoni acuti come sibilanti o fruscii.
 - Filtro di riduzione del rumore: Rimuove rumori di fondo costanti (es. ventilatori, ronzio), migliorando la chiarezza della voce.
 - Filtro di normalizzazione del volume: Regola l'audio per avere un volume omogeneo, evitando picchi troppo alti o parti troppo basse.
- Whisper: Whisper AI è un sistema sviluppato da OpenAI che

trascrive automaticamente l'audio in testo. Funziona ascoltando un file audio, lo divide in piccoli pezzi, li analizza trasformandoli in immagini di spettro (spettrogrammi), e poi li "traduce" in parole usando una rete neurale. È molto bravo a riconoscere voci anche con accenti diversi, rumori di fondo o in lingue diverse. Può anche tradurre parlato da una lingua all'altra. E' utile ad esempio per trascrivere interviste, video, dettature o qualsiasi registrazione, ed è open source, quindi lo si può installare e usare direttamente sul proprio computer.

3.2.2 NER e LLM

Il sistema utilizza un modello di **Large Language Model (LLM)** per generare report clinici strutturati, ovvero la **scheda di ammissione al Pronto Soccorso** o il **referto di una visita medica**, a seconda della modalità operativa selezionata.

Poiché l'intero sistema esegue **localmente**, l'utilizzo di modelli molto profondi è limitato, il che rappresenta una sfida nella generazione di testi clinici accurati. Per affrontare efficacemente questa limitazione, il sistema fornisce alla LLM il contesto e sull'uso della **prompt engineering**.

1. In fase di generazione, viene effettuata un'estrazione preliminare delle entità cliniche tramite un modello NER (*DeepMount/universal_ner_ita*). Questo consente alla LLM di compren-

dere meglio il contenuto semantico del testo, migliorando la coerenza del referto prodotto.

2. Alla LLM viene fornita una struttura predefinita (template JSON) e un esempio concreto, per guidare la generazione verso un output compatibile con i requisiti clinici.
3. I prompt sono stati progettati in modo mirato per:
 - Minimizzare il rischio di allucinazioni.
 - Guidare la LLM verso un output JSON strutturato e valido.
 - Adattarsi dinamicamente alla modalità operativa del sistema (es. PS o visita ambulatoriale).
 - Produrre un testo esclusivamente in italiano a prescindere dalla lingua del testo fornito.

I prompt realizzati ad hoc in base al documento da generare sono i seguenti:

Prompt scheda di pronto soccorso

```
"Sei un medico in pronto soccorso. Ricevi un testo ↵
  ↳ discorsivo (esempio trascrizione verbale) e devi ↵
  ↳ generare una scheda di ammissione del paziente al ↵
  ↳ pronto soccorso."
```

```
"La scheda deve essere in italiano formale, chiara e ben↵
  ↳ strutturata in formato JSON. Non inserire dati ↵
  ↳ inventati, attieniti a quelli forniti nel testo."
```

```
"Se una sezione e' assente, scrivi 'N/A'. Ecco un ↵
    ↪ esempio di testo che potresti ricevere:"
f"{esempio_trascrizione}"

"Ecco un esempio di output relativo alla trascrizione ↵
    ↪ sopra riportata:"
f"{esempio_output}"

"Ecco lo schema che devi seguire per generare la scheda ↵
    ↪ di ammissione al pronto soccorso:"
f"{json.dumps(esempio_scheda, ensure_ascii=False, indent↵
    ↪ =2)}"

f"Le seguenti entita' sono state riconosciute nel testo ↵
    ↪ e possono aiutarti a completare la scheda:\n{↵
    ↪ entities}\n\n"

"Rispondi solo con un JSON valido, non scrivere ↵
    ↪ introduzioni, commenti o spiegazioni."
```

Prompt referto ospedaliero

```
"Sei un assistente clinico. Ricevi un testo discorsivo (↵
    ↪ esempio trascrizione verbale) e devi generare un ↵
    ↪ referto medico per un paziente che hai visitato."

"La scheda deve essere in italiano formale, chiara e ben↵
    ↪ strutturata in formato JSON. Non inserire dati ↵
    ↪ inventati, attieniti a quelli forniti nel testo."

"Se una sezione e' assente, scrivi 'N/A'. Ecco un ↵
    ↪ esempio di testo che potresti ricevere:"
f"{esempio_trascrizione}"

"Ecco un esempio di output relativo alla trascrizione ↵
    ↪ sopra riportata:"
```



```
f"{esempio_output}"

"Ecco lo schema che devi seguire per generare la scheda ←
    ↳ di ammissione al pronto soccorso:"

f"{json.dumps(esempio_referto, ensure_ascii=False, ←
    ↳ indent=2)}"

f"Le seguenti entita' sono state riconosciute nel testo ←
    ↳ e possono aiutarti a completare la scheda:\n{←
    ↳ formatted_entities}\n\n"

"Rispondi solo con un JSON valido, non scrivere ←
    ↳ introduzioni, commenti o spiegazioni."
```

Output della LLM

Per garantire che l'output della LLM sia effettivamente un JSON valido, sono state implementate diverse misure:

- **one-shot learning**: con esempi completi di input/output.
- **extract_json_from_response**: estrae solo la porzione JSON, rimuovendo testo introduttivo o altri artefatti.
- **fix_json_format**: utilizza **repair_json** per correggere eventuali errori di formattazione.
- **check_json_format**: verifica finale per assicurarsi che la struttura sia ben formata.

Dopo aver effettuato il controllo della struttura del report generato e corretto eventuali errori, questo viene integrato con i seguenti elementi:

- Anagrafica del medico
- Anagrafica del paziente
- Referto generato (testo strutturato)
- ID univoco (comune con la trascrizione per tracciabilità)
- Stato di validazione (**False**)

Il referto completo viene quindi salvato in un file JSON all'interno della cartella *assets/*, e successivamente inserito nella collection **Clinical Reports** del database, permettendone così la validazione da parte dell'utente tramite dashboard. Per rendere il sistema scalabile e adattabile alle risorse hardware disponibili:

- Se è disponibile una GPU CUDA:
 - Viene scaricato il modello Mistral-7B da Hugging Face (DeepMount).
 - Il modello viene quantizzato localmente usando bitsandbytes in base alle capacità del dispositivo.
- Se si opera su CPU:
 - Viene scaricata direttamente la versione quantizzata (GGUF) del modello, già pronta all'uso.
 - Il modello è gestito tramite ctransformers.

Pipeline Manager

Questo componente ha il compito di ****orchestrare l'intero flusso di esecuzione del sistema****.

Una volta acquisito il ****percorso del file audio**** registrato come input, viene eseguita la ****trascrizione automatica**** del contenuto.

Successivamente, viene generato un ****ID univoco**** che permette di associare in modo univoco la trascrizione al referto clinico e viceversa.

La trascrizione, insieme al relativo ID, viene quindi salvata nella collection 'Transcription' del database.

A partire dalla trascrizione prodotta, il sistema:

- Estrae l'anagrafica del paziente
- Anonimizza i dati sensibili, in modo che la LLM operi su contenuti privi di informazioni personali.

La pipeline prosegue poi con:

- La generazione del referto clinico strutturato tramite LLM
- Il salvataggio del referto nella collection Clinical Reports del database.

3.3 Database

Il componente **Database** è un pilastro fondamentale del sistema, progettato per garantire la persistenza dei dati e una gestione efficiente delle informazioni. La sua struttura si articola su due macro-elementi distinti, ciascuno con un ruolo specifico e ottimizzato per le proprie funzionalità:

- **MongoDB:** Questo *database NoSQL* orientato ai documenti è la scelta primaria per l'archiviazione dei referti, delle trascrizioni e di tutti i dati non relazionali.
 - **Cos'è e come funziona:** MongoDB memorizza i dati in documenti in formato *BSON* (una rappresentazione binaria di JSON), raggruppati in collezioni. A differenza dei database relazionali, non richiede uno schema fisso, offrendo una grande flessibilità e scalabilità orizzontale. È particolarmente adatto per dati complessi e in evoluzione.
 - **Collezioni Principali:** All'interno di MongoDB, i dati sono organizzati in tre collezioni principali:
 - * **medical_operators:** Contiene tutte le informazioni relative agli utenti del sistema, inclusi i medici e il personale sanitario.
 - * **transcriptions:** Utilizzata per salvare le trascrizioni generate dal servizio Whisper.

- * **clinical_reports**: Archivia i referti medici completi e le informazioni dettagliate sui pazienti.
- **Perché utilizzarlo**: La sua flessibilità di schema (*schema-less*) permette di adattarsi rapidamente a cambiamenti nei requisiti dei dati senza interruzioni. La sua architettura distribuita consente una facile scalabilità per gestire grandi volumi di dati e traffico elevato, garantendo alta disponibilità e prestazioni.
- **Code Redis**: Affiancate a MongoDB, le *code Redis* (*Remote Dictionary Server*) sono impiegate per la gestione asincrona delle operazioni e la memorizzazione temporanea di dati.
 - **Cosa sono e come funzionano**: Redis è un *data structure store in-memory* che può essere usato come database, cache e message broker. Opera mantenendo i dati principalmente in RAM, garantendo così velocità di accesso estremamente elevate. È ampiamente utilizzato per implementare code di messaggi (ad esempio, tramite liste o Pub/Sub) e per la gestione di cache ad alta velocità.
 - **Perché utilizzarle**: L'utilizzo di Redis come coda permette di disaccoppiare i processi, migliorando la reattività del sistema e gestendo picchi di carico. Le operazioni pesanti possono essere messe in coda ed elaborate in back-

ground senza bloccare l'interfaccia utente. La sua natura in-memory offre una latenza bassissima, ideale per operazioni che richiedono risposte quasi istantanee.

3.3.1 Popolamento del Database

Per la popolazione del database i passi principali sono stati i seguenti:

- Switch del sistema di acquisizione vocale e generazione del referto con un sistema posto in cloud ed offerto tramite chiamate API dal servizio Groq Cloud.
- Tramite il sistema cloud e attraverso una fase di prompt engineering, viene generato il dataset dei referti clinici associati a medici e pazienti.
- Una volta generati i referti come file JSON, il sistema ne estrae le informazioni, inserisce i referti, ottiene le anagrafiche dei medici associate al referto e ne registra gli account.
- In questo modo, si dispone di un playground del sistema completamente funzionante e pronto ad essere utilizzato.

3.3.2 Analytics per l'Amministratore

Il sistema offre all'amministratore una suite di **analytics avanzate** per monitorare le performance operative e supportare decisioni strate-

giche nella gestione dell'ospedale. Queste analisi sono suddivise in tre categorie principali:

- **Analitiche Temporal**i: Queste metriche permettono di effettuare analisi di serie storiche sull'andamento dei referti. L'amministratore può selezionare un intervallo di tempo specifico per visualizzare:
 - La *media dei referti* giornaliera nell'intervallo selezionato.
 - Il *numero massimo di referti* registrati in una singola giornata.
 - Il *numero totale di decessi* avvenuti in quel periodo.
 - Un *linear plot* che illustra l'andamento giornaliero dei referti nel tempo, offrendo una chiara visualizzazione delle tendenze.
- **Analitiche sui Medici**: Focalizzate sulle performance individuali e la distribuzione del carico di lavoro del personale medico:
 - La *top 10 dei medici con il maggior numero di referti* emessi, utile per identificare il personale più oberato e bilanciare il carico di lavoro.
 - La *distribuzione dei referti per reparto e fascia oraria*, fornendo una panoramica del carico di lavoro in specifiche aree e periodi della giornata.

- **Analitiche Complesse:** Progettate per identificare colli di bottiglia e inefficienze nella distribuzione del carico ospedaliero:
 - Una *calendar heatmap* che visualizza la distribuzione del numero di referti per reparto e giorno della settimana, comprendo gli ultimi 365 giorni, per evidenziare pattern stagionali o ricorrenti.
 - Un’analisi temporale per reparto che aggrega dati su tre variabili chiave per gli ultimi 365 giorni: il *numero totale di referti*, la *media dei referti giornaliera* e il *numero di giorni di attività* del reparto.

3.4 Dashboard

La dashboard rappresenta l’interfaccia principale attraverso cui gli utenti sanitari interagiscono con il sistema di gestione dei referti clinici. È progettata per essere intuitiva, sicura e accessibile da diverse categorie di operatori, distinguendo chiaramente tra utenti base (medici, infermieri, operatori sanitari) e utenti con privilegi avanzati (amministratori di sistema).

Autenticazione e Gestione degli Utenti

Tutti gli utenti accedono al sistema tramite un modulo di **registrazione e login** sicuro. I profili utente sono differenziati per ruolo:

- **Utenti base** (es. medici e infermieri) possono registrarsi autonomamente.
- **Amministratori** non possono autogenerare un account: per motivi di sicurezza, la loro creazione avviene esclusivamente tramite un altro amministratore già presente nel sistema.

Funzionalità Utenti Base

Gli utenti base hanno a disposizione un insieme di strumenti fondamentali per la gestione quotidiana dei referti clinici:

- Possono **generare un nuovo referto** cliccando sull'apposito bottone nella dashboard, che invoca una pipeline LLM (Large Language Model) per l'elaborazione automatica di testi clinici.
- Hanno accesso a tutti i referti dei pazienti associati al proprio profilo, con la possibilità di **visualizzarli, modificarli, eliminarli** o **scaricarli** in formato standard.
- Possono effettuare **query di filtraggio avanzato** su tali referti, ad esempio per data, diagnosi, codice fiscale del paziente o stato della compilazione.

Funzionalità Amministratori

Gli amministratori, oltre alle funzionalità base, dispongono di strumenti avanzati di supervisione e analisi:

- Possono **eseguire query** sui referti di tutti i pazienti dell'ospedale.
- Accedono a una sezione dedicata denominata **Analytics**, che fornisce:
 - **Statistiche temporali** sull'attività clinica (es. numero di referti per giorno/settimana/mese).
 - **Query aggregate per medico**, utili per monitorare la produttività o l'efficienza dei professionisti.
 - **Analisi complesse** su dati da più reparti, utili per individuare **colli di bottiglia** nei flussi di lavoro.

3.5 Dockerization

3.5.1 Dockerizzazione del sistema

Per garantire la portabilità, la scalabilità e la replicabilità del sistema, è stata adottata una soluzione basata su Docker. L'intero ecosistema applicativo è suddiviso in più container, ognuno responsabile di un componente specifico, come il backend, il controller logico, la dashboard, i database e le code di messaggistica.

Docker consente di eseguire ogni componente in ambienti isolati ma comunicanti attraverso una rete interna definita (mynetwork). Questo approccio elimina problemi legati alla configurazione dell'ambiente locale, garantisce comportamenti consistenti in fase di sviluppo, test e

produzione, e rende più semplice la gestione delle dipendenze.

3.5.2 Struttura del Dockerfile

Il file Dockerfile definisce un'immagine di base leggera (python:3.10-slim) su cui viene costruito l'intero ambiente Python. Include:

- Configurazione della lingua in italiano (utile per output e logging in lingua).
- Installazione delle dipendenze via requirements.txt.
- Download del modello linguistico spaCy per la lingua italiana (it_core_news_sm).
- Copia del codice applicativo e configurazione dello script di avvio (entrypoint.sh).
- Esposizione delle porte necessarie per ogni componente (8001, 8003, 8501).

Questo Dockerfile è condiviso da più servizi per garantire consistenza tra backend, controller, dashboard e init script.

3.5.3 Docker Compose: orchestrazione multi servizio

Il file docker-compose.yml definisce e orchestra l'intero ecosistema applicativo. I principali servizi inclusi sono:

- mongo: database NoSQL per la memorizzazione dei dati clinici.
- redis: sistema di messaggistica e caching in tempo reale.
- backend: API REST per la gestione dei referti clinici.
- controller: servizio per l'elaborazione e validazione dei dati.
- dashboard: interfaccia utente interattiva sviluppata con Streamlit.
- init-db: script che inizializza il database alla partenza del sistema.

Ogni servizio è containerizzato e collegato tramite una rete virtuale comune. I volumi persistenti garantiscono la conservazione dei dati Mongo anche in caso di riavvio del container.

Chapter 4

Testing

Il processo di testing per l'applicazione "Clinical Report AI Assistant" ha compreso diverse fasi cruciali per garantirne la robustezza e la funzionalità. In primo luogo, è stato condotto un approfondito **Integration Testing**, focalizzato sulla verifica dell'interazione tra i moduli chiave: `Pipeline_Manager` e `Database`. Questo tipo di test è stato implementato attraverso l'esecuzione di chiamate a funzioni API esposte dal controller dell'applicazione, permettendo di accertare che i dati venissero correttamente elaborati e persistiti tra i diversi componenti del sistema. In aggiunta, sono stati effettuati **test grafici** volti a convalidare il funzionamento basilare dell'applicazione dal punto di vista dell'utente. Questi test hanno previsto la registrazione delle interazioni con l'interfaccia grafica e il salvataggio dei relativi file di log e screenshot all'interno della cartella `Tests` presente nel repository GitHub del progetto. Questa metodologia ha permesso di documentare

visivamente il comportamento dell'applicazione e di identificare eventuali anomalie nell'esperienza utente.

4.1 Integration testing

4.1.1 Test di Pipeline Manager

Il modulo Pipeline Manager è fondamentale per l'orchestrazione dei processi di generazione dei report. Per garantirne il corretto funzionamento, sono stati definiti due test chiave.

Il Test 1: Inizializzazione della Pipeline si concentra sulla fase di configurazione iniziale del modulo. L'obiettivo è istanziare la classe **Pipeline_Manager** e verificare che l'oggetto creato non sia nullo, confermando così che l'inizializzazione avvenga senza errori.

Successivamente, **il Test 2:** New Report mira a validare la capacità del sistema di generare nuovi report. Questo test richiede come prerequisito l'inizializzazione del server controller e del backend per simulare un ambiente operativo completo. Una volta soddisfatti i prerequisiti, il test chiama la funzione **new_report** dal controller e verifica che l'output sia un **report_id** valido, assicurando che la pipeline sia in grado di avviare correttamente la creazione di un nuovo report.

```

def new_report(self, filename, anagrafica_medico=None):
    """
    Create a new report in the database.
    """
    func_mode = self.pipeline_manager.function_mode
    response = requests.post(
        url=f"{self.controller_url}/new_report",
        json = {
            "text": filename,
            "anagrafica_medico": anagrafica_medico,
            "function_mode": func_mode,
        })

    assert response.status_code == 200, "Failed to create a new report"
    if response.status_code == 200:
        report = response.json()

    print("Test report creation successful:")

    return report["report"].get("_id")

def test_pipeline_initialization(self):
    """
    Test to ensure the pipeline initializes correctly.
    """
    assert self.pipeline_manager is not None, "PipelineManager should be initialized"
    assert self.db is not None, "Database connection should be established"

    print("Test pipeline initialization successful.")

def test_pipeline_run(self):
    """
    Test to ensure the pipeline runs without errors.
    """

    # Ipotizzando l'esistenza di un file audio di test
    audio_file_path = "./assets/audios/test_audio.wav"

    id_report = self.new_report(audio_file_path, self.anagrafica_medico)

    assert id_report is not None, "Pipeline should return a valid report ID"
    return id_report

```

4.1.2 Test del Database

Il modulo Database è cruciale per la persistenza dei dati e la gestione dei report. Per validarne l'affidabilità, sono stati implementati due test specifici. Il primo, il Test di Connessione al Database, ha lo scopo di verificare che il modulo possa stabilire una connessione stabile e autenticata con il database. Questo test non solo assicura che il server del database sia raggiungibile, ma anche che le credenziali e le configurazioni di rete siano corrette, garantendo una base solida per tutte le operazioni successive. Il secondo test, l'Inserimento di un Report nel Database, si concentra sulla funzionalità di scrittura dei dati. Dopo aver stabilito una connessione valida, questo test simula l'inserimento di un nuovo record di report nel database, verificando che l'operazione venga completata con successo e che il report venga correttamente registrato. Insieme, questi test assicurano che il modulo Database sia pienamente operativo sia nella fase di connessione che in quella di manipolazione dei dati.

4.2 UI Testing

Infine, per quanto riguarda la User Interface (UI), l'approccio di testing si è focalizzato sulla copertura dei cammini. Nonostante la complessità che può derivare dalla miriade di interazioni possibili, abbiamo privilegiato la verifica dei cammini più importanti e critici che un


```
def test_db_connection(self):
    """
    Test to ensure the database connection is established.
    """
    assert self.db is not None, "Database connection should be established"

def insert_report_test(self):
    """
    Test to retrieve a report from the database.
    """

    report_id = "test_report_id"
    report = {
        "dati medico": self.anagrafica_medico,
    }

    inserted_id = self.db.insert_clinical_report(report_id, report)

    assert inserted_id is not None, "Report should be inserted successfully"

    retrieved_report = self.db.get_report_by_id(inserted_id)

    assert retrieved_report is not None, "Report should be retrievable from the database"
```

utente può intraprendere. Questi test sono stati condotti attraverso la registrazione con un sistema di acquisizione grafica, permettendoci di catturare e analizzare in dettaglio ogni interazione e reazione della UI. Questo ci ha permesso di assicurarci che le funzionalità principali siano accessibili, che il flusso di navigazione sia intuitivo e che non ci siano blocchi in punti cruciali dell'esperienza utente, garantendo così la stabilità e l'usabilità dell'interfaccia.

OSS: le registrazioni dei test effettuati sono disponibili nel branch "Testing" della repository github

Chapter 5

Conclusioni

Il sistema è stato progettato con una visione a lungo termine, che prevede l'adattabilità all'evoluzione tecnologica e la possibilità di utilizzo anche su device mobili, come smartphone e smartwatch, che consentirebbe di aumentare notevolmente la portabilità del sistema e semplificherebbe il lavoro del personale sanitario, rendendolo meno dipendente dal PC durante le visite ai pazienti.

Un'ulteriore evoluzione possibile riguarda l'introduzione del Federated Learning, una tecnica di apprendimento automatico che consente l'addestramento del modello direttamente nei dispositivi locali (o su server interni), utilizzando dati anonimizzati dei pazienti senza doverli trasferire esternamente.

Questa strategia migliorerebbe significativamente l'accuratezza del sistema, rendendo possibile:

- L'impiego di modelli più leggeri e ottimizzati per l'esecuzione

locale.

- L'adattamento progressivo del modello alle specifiche esigenze cliniche.
- Il mantenimento di alte performance senza sacrificare la privacy né aumentare i requisiti hardware.

Al fine di rafforzare ulteriormente la qualità del testo generato, si potrebbe poi prevedere l'integrazione di un sistema RAG (Retrieval-Augmented Generation), capace di identificare automaticamente trascrizioni simili a quella fornita al sistema, estrapolare il referto corrispondente già validato da personale medico e utilizzarli in fase di generazione. Ciò consentirebbe di migliorare l'efficacia del one-shot learning, rendendo la generazione più precisa e aderente agli standard clinici.

Dal punto di vista della gestione dei dati, il sistema potrebbe essere potenziato anche includendo immagini diagnostiche, come ecografie o radiografie (X-ray), ampliando così le tipologie di query eseguibili sul database e le funzionalità disponibili per gli utenti.

Questo aprirebbe la strada all'integrazione di modelli AI avanzati, come quelli per la instance segmentation, che potrebbero essere utilizzati anche in contesti formativi.

Si potrebbe introdurre, infatti, un nuovo tipo di utente, lo studente, che avrebbe accesso a funzionalità didattiche potenziate, inclusa la possibilità di sperimentare con modelli di intelligenza artificiale per apprendere nozioni cliniche su dati realistici e casi d'uso simulati.

Si conclude la trattazione, infine, proponendo uno switch dell'architettura utilizzata per le code Redis da una coda classica a una architettura Publisher/Subscribe, che aumenterebbe ancora di più l'efficienza del retrieval dei dati.

